Tree based methods

Part 1 - Getting to know the data

Run the python code in Part 1 and answer the following questions

1.1 How many predictors and how many data points does the dataset have?

1.2 The response variable is `Class`, how many categories does it have?Is the dataset balanced?

1.2 Looking at the generated plots, which variables seem most useful?

Part 2: A first, simple tree

2.1 Which variable is used to split the data? What is the threshold value used in the split and what does this threshold mean in practice?

2.2 How would you explain this model to someone with no statistics background?

2.3 A bean has Area = 50,000. What is the predicted bean variety? Another bean has Area = 80,000. What is the predicted bean variety? What happens if two beans have the same value of Area but very different values of Compactness?

2.4 Do you think that the model is overfitting or underfitting?

Part 3: A deeper tree

3.1 Predict the class for the following beans based on the tree with depth = 6 created in Part 3 of the code:

Bean A

- **Area:** 40,000
- **Perimeter:** 800
- **MajorAxisLength:** 300
- **MinorAxisLength:** 180
- **Compactness:** 0.75

Predicted variety: _____

Bean B

- **Area:** 90,000
- **Perimeter:** 1200
- **MajorAxisLength:** 450
- **MinorAxisLength:** 250
- **Compactness:** 0.85

Predicted variety: _____

Predicted species: _____

3.2 Which variable appears most often in the splitting rules? What does this suggest about its usefulness for classifying bean species?

3.3 Do you think this tree can be generalized on new bean data? Explain your reasoning and discuss whether the tree might be overfitting.

Part 4 - Tuning the tree depth

In part 4 the code performs a cross-validation experiment to tune the tree depth parameter.

4.1 How many folds are used in the cross-validation procedure? How can you tell from the code?

The table produced reports the mean accuracy for each value of the tree depth. The accuracy is defined as:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total numnef of predictions}}$$

4.2 Why does the output report the "Mean CV Accuracy" rather than a single accuracy value? What does the reported standard deviation tell us?

4.3 Does performance improve indefinitely as depth increases? Why might very deep trees perform worse than shallow tree? Which depth value would you choose?

Part 5 - Fitting a Randon Forest

Run part of the Python code. Here we fit a random forest to the beans data.

5.1 How many trees are included in this random forest? How can you tell from the code?

5.2 Before evaluating the model, would you expect the random forest to perform better or worse than a single decision tree? Explain your reasoning.

5.3 What is one advantage and one disadvantage of a random forest compared with a single decision tree?

Part 6: Cross validation for random forests

Run the code in Part 6. This is a 5-fold cross validation to tune two of the main parameters in a random forest: the tree depth and the number of trees in the forest.

Note: the code takes some seconds to be finished.

6.1 Based on these results, what values of `n_estimators` and `max_depth` would you choose for the final model?

Part 7: Testing on the test set

7.1 If your goal were prediction, which model would you choose: the best decision tree or the best random forest? Explain your answer using the test-set results.

7.2 What is the accuracy of the selected model on the test set? How does this compare with the cross-validation accuracy obtained during model selection?

7.3 If your goal were interpretability, which model would you choose? Why?

7.4 Examine the confusion matrix.

- Which bean variety is most often classified correctly?
- Which bean variety is most often misclassified?
- Which pair of bean varieties appears most difficult to distinguish?